

Density Estimation - Examples in R

B.A. Hartlaub and A.S. Wagaman

Note: The code here is kept consistent with the code in Hollander, Wolfe, and Chicken, and is not updated to ggplot2, though we suspect all the same options are available, and may update this at a later time to reflect that.

Density Estimation

When examining a data set, understanding the distribution of the variables is important for choosing statistical methods and understanding what you have to work from. Often, we examine distributions of individual variables (univariate distributions). A principal way of doing that is with visuals, such as histograms. But histograms have options. They are examples of density estimation techniques. The default presentation isn't the only option you have!

Histograms and Their Options + ECDFs

Egyptian Skulls Data

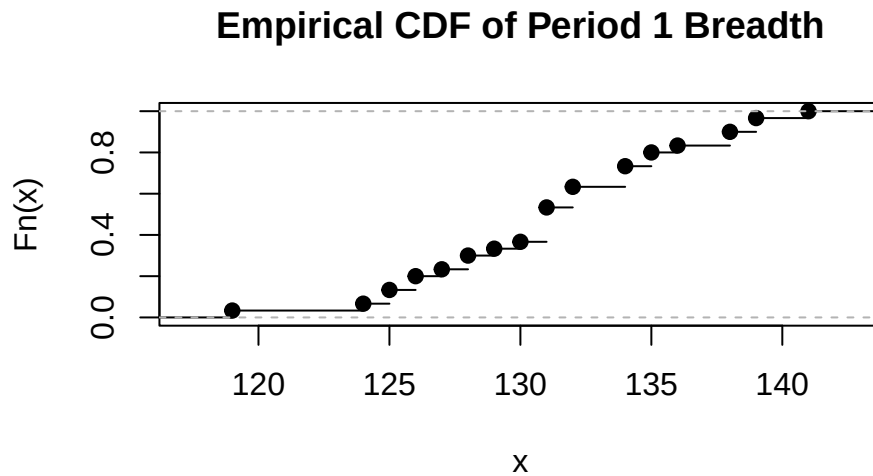
For data, we'll look at a data set on Egyptian skulls. The skulls are from 3 different time periods (1 = 4000 BC, 2 = 3300 BC, and 3 = 1850 BC). Measurements are all in mm for maximum breadth of skull (mm), basibregmatic height (bbheight), basialveolar length (bvlenght), and nasal height (nasal). It may be interesting to compare across periods, so we created data sets for each, plus we still have the overall data set.

```
skulls <-  
  read.table("https://awagaman.people.amherst.edu/nonparametricstats/egyptskulls.txt",  
            h = T)  
p1 <- filter(skulls, period == 1)  
p2 <- filter(skulls, period == 2)  
p3 <- filter(skulls, period == 3)
```

Plotting Empirical CDFs

First, before we look at the histogram options, we want to show you that we can plot the empirical cdfs (empirical cumulative distribution functions) that were being compared via the Kolmogorov-Smirnov tests. Overlaying is difficult, as the `ecdf` function returns a function object, but you can at least now look at univariate empirical CDFs if you like. The plots are in base R.

```
plot(ecdf(p1$breadth), main = "Empirical CDF of Period 1 Breadth")
```



How does that compare to something generated that is similar from the optimal normal distribution?

```
favstats(~ breadth, data = p1)
```

```
## min Q1 median Q3 max mean sd n missing
## 119 128 131 134.75 141 131.3667 5.129249 30 0
```

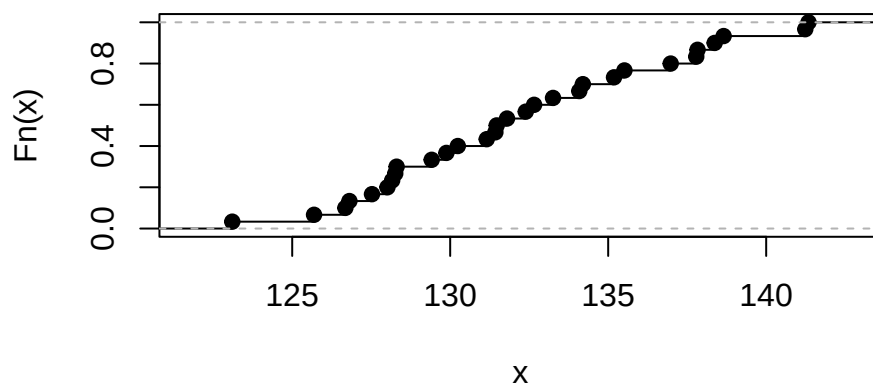
```
set.seed(225)
```

```
# used 30 points to match the 30 skulls from the time period
```

```
normtemp <- rnorm(30, 131.3667, 5.129249)
```

```
plot(ecdf(normtemp),
     main = "Empirical CDF of 30 Obs at Random from Normal(131.3667,5.129249)")
```

irical CDF of 30 Obs at Random from Normal(131.3667,



The difference you see here is that the real data were rounded, so there are ties and it can jump up due to multiple observations at a single x value, and thus its CDF doesn't have as many jumps as the simulated one. That's very common with discrete data, and one reason that many density estimation techniques have ways

to smooth their estimates.

Histograms and Their Options

Now, on to the histograms and their options.

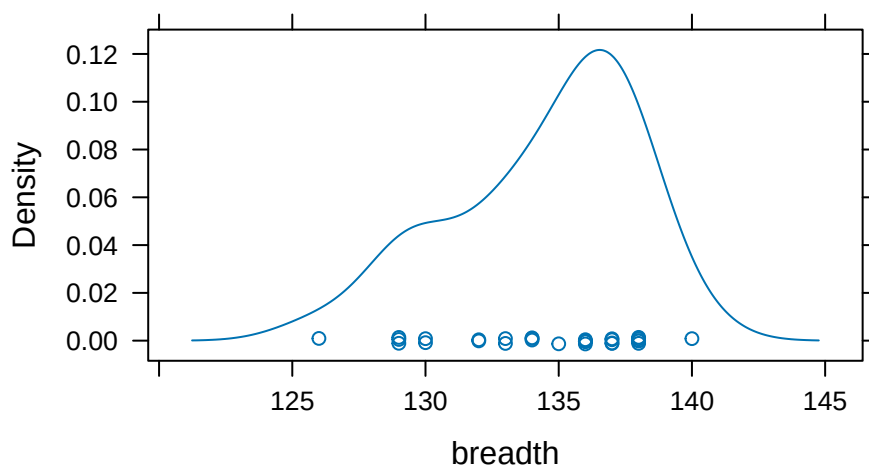
```
# ?hist #lets you look at hist options in base R  
# ?histogram #lets you look at histogram options in mosaic
```

We can use `histogram` with all the breaks options mentioned for `hist`. Additionally, you can swap between a density estimate and a frequency plot by specifying the `type`. The default type is density.

From exploring the Egyptian skull breadth distributions, we might think the density plot for the breadth in period 3 had perhaps 2 peaks in it, so lets see what the different histogram options give us for that. Note that if not using the default, the Y axis changes to Percent of Total. You can put it to density with the `type` option, as shown below.

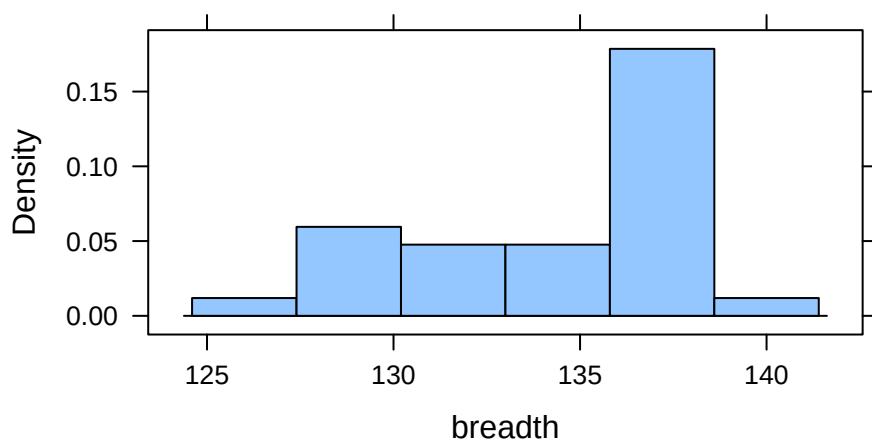
We will use the `densityplot` and `histogram` options in `mosaic` rather than converting to `ggplot2` because they allow us to be consistent with the text.

```
densityplot(~ breadth, data = p3)
```



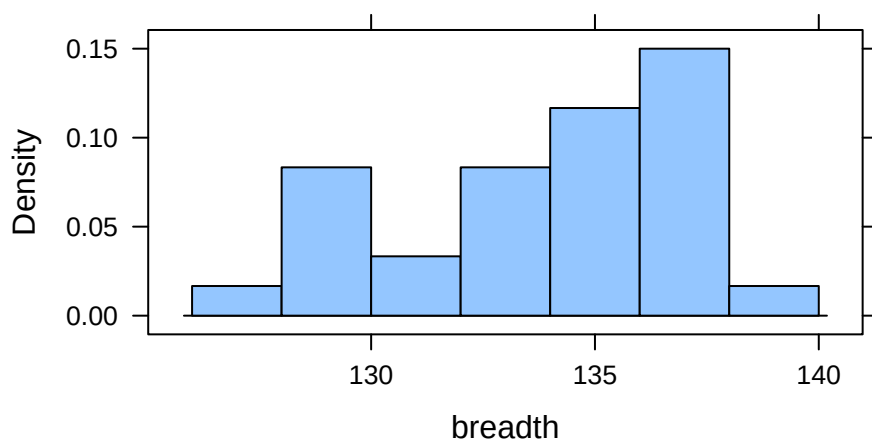
```
histogram(~ breadth, data = p3, main = "Default Histogram") # break default
```

Default Histogram

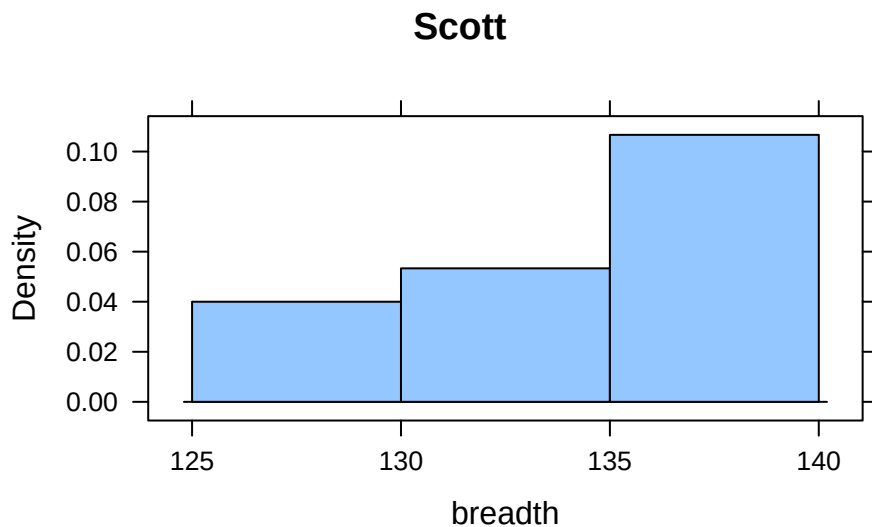


```
histogram(~ breadth, data = p3, breaks = "Sturges", main = "Sturges", type = "density")
```

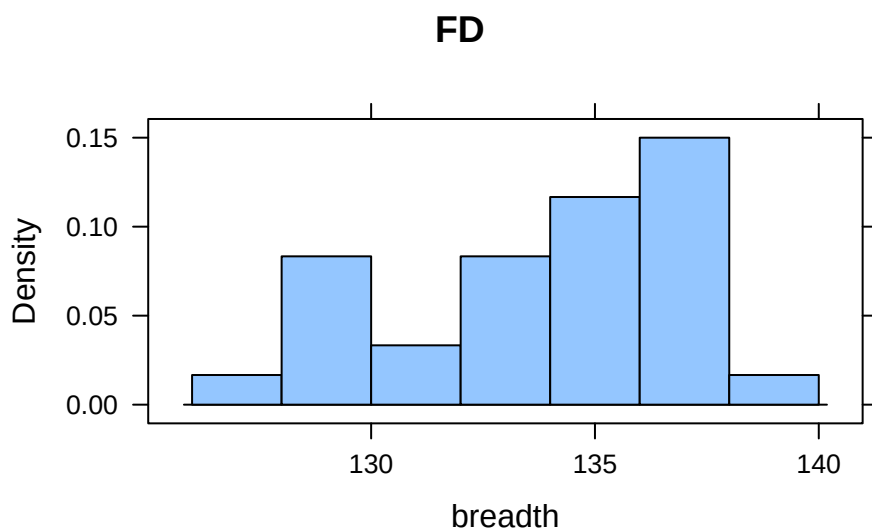
Sturges



```
histogram(~ breadth, data = p3, breaks = "Scott", main = "Scott", type = "density")
```



```
histogram(~ breadth, data = p3, breaks = "FD", main = "FD", type = "density")
```



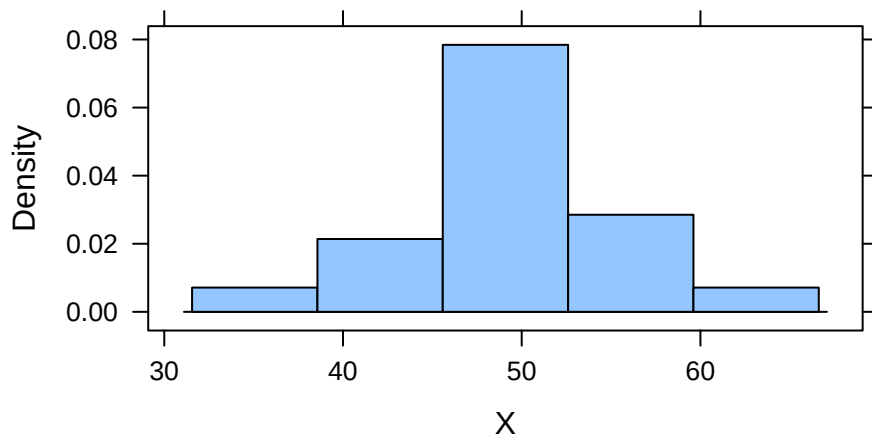
Recall - these are all created from the SAME data. How different do the distributions look? It's hard to get the h values from the plots, but we can figure out the m values (number of intervals) and then solve for h if we wanted. (Notation from the Hollander, Wolfe, and Chicken text, h can be span or bandwidth depending on what procedure you are referring to.)

For the default, $m=6$, and there appears to be one clear peak, with perhaps a second one to the left. For Sturges, $m=7$, and the two peaks are a little more clear, one still dominates. For Scott, $m=3$, and the plot is mostly useless for seeing a second peak. For FD, $m=7$, and the plot looks identical to that from Sturges (for this variable).

How well do the histograms do with normally distributed RVs? We can simulate to look.

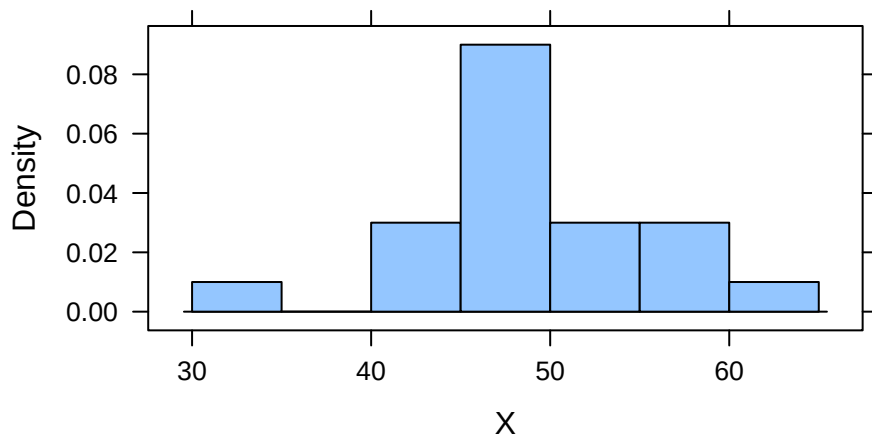
```
set.seed(20) # to make results reproducible
X <- rnorm(20, 50, 6)
histogram(~ X, main = "Default") # break default
```

Default

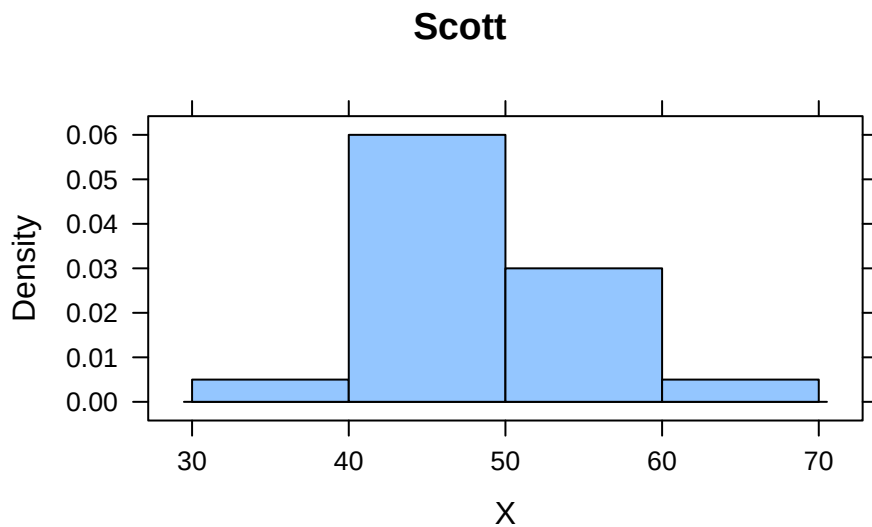


```
histogram(~ X, breaks = "Sturges", main = "Sturges", type = "density")
```

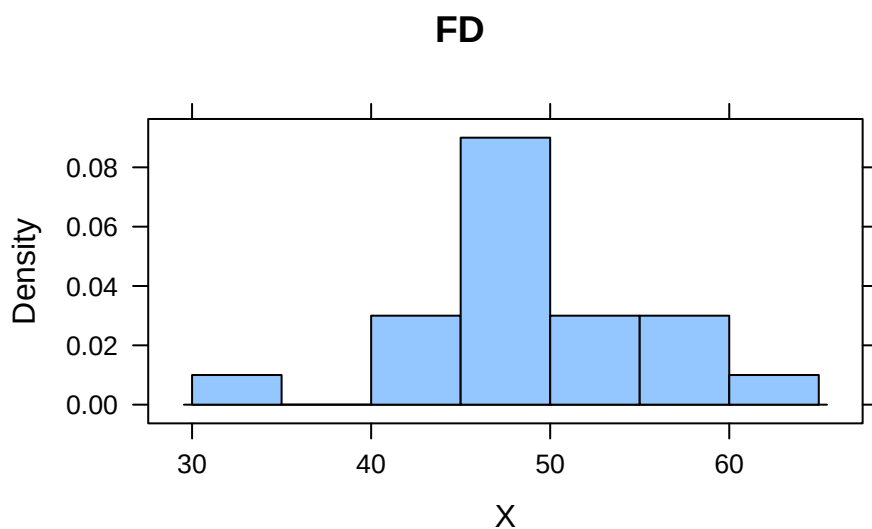
Sturges



```
histogram(~ X, breaks = "Scott", main = "Scott", type = "density")
```



```
histogram(~ X, breaks = "FD", main = "FD", type = "density")
```

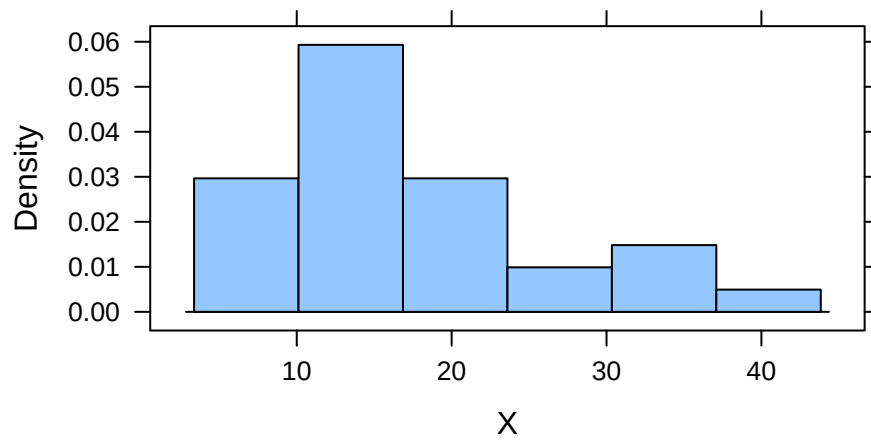


For this particular example, Sturges and FD seem to yield the same plot again.

What about with something really skewed? Remember we suspect Scott's method will not do so well in providing a good density estimate (though it hasn't been that awesome up till now anyway).

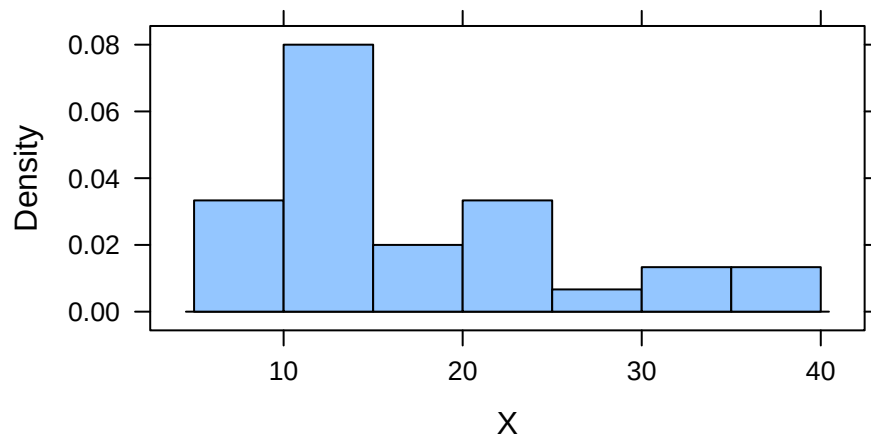
```
set.seed(60) # to make results reproducible
X <- rgamma(30, 3, 1/6) # gamma with mean 18, right skew
histogram(~ X, main = "Default") # break default
```

Default



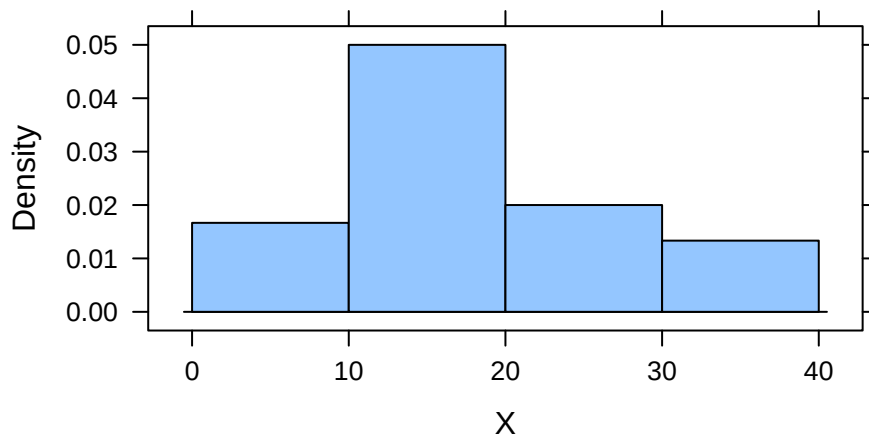
```
histogram(~ X, breaks = "Sturges", main = "Sturges", type = "density")
```

Sturges



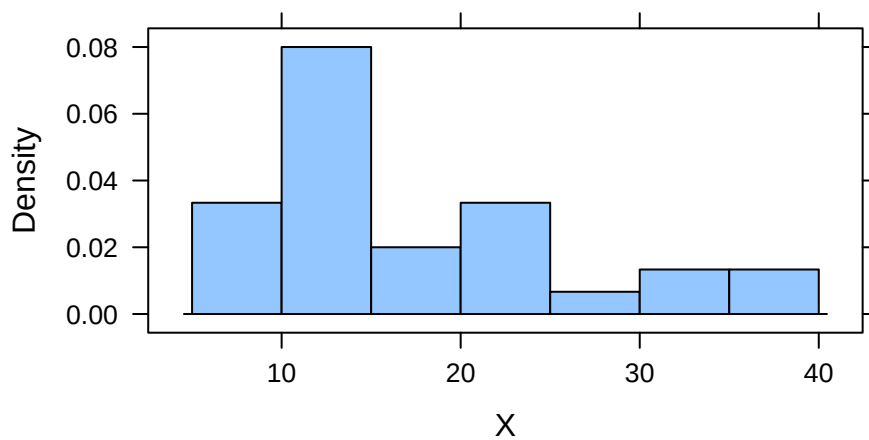
```
histogram(~ X, breaks = "Scott", main = "Scott", type = "density")
```


Scott



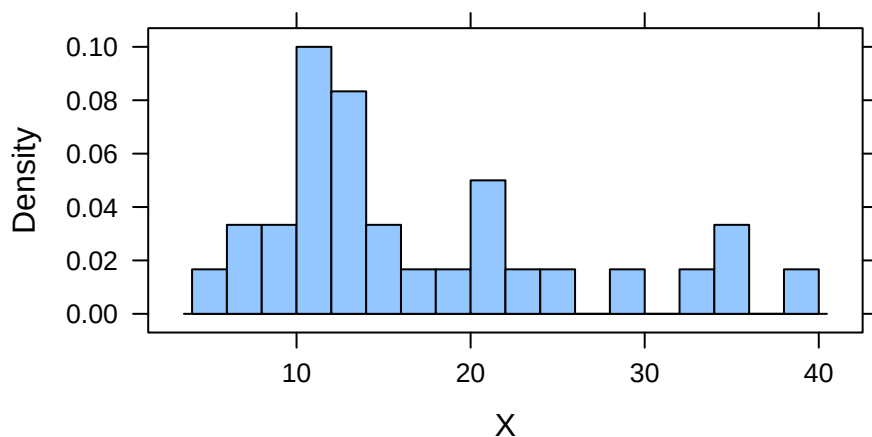
```
# you can write out Freedman-Diaconis if you want  
histogram(~ X, breaks = "FD", main = "FD", type = "density")
```

FD



```
# choose our own number of breaks  
histogram(~ X, breaks = 15, main = "15 Breaks", type = "density")
```

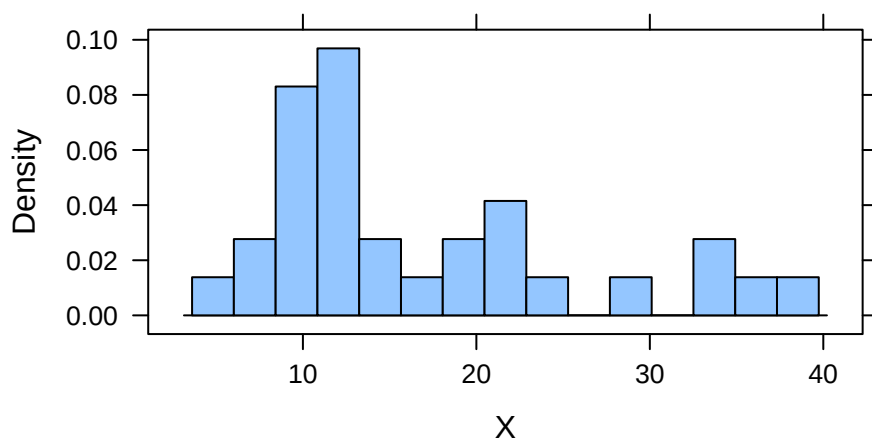
15 Breaks



Sometimes the breaks don't seem to give us the picture we want. We can set an approximate the number of bins too, using `nint`.

```
histogram(~ X, nint = 15, main = "Approx. 15 Bins", type = "density")
```

Approx. 15 Bins



How can we get the h values? We can solve for h using `favstats`, and estimate m based on that, assuming the endpoints for the histogram are the min and max of the data (usually, we go past that, so these m values are usually too low). Here is an example.

```
favstats(~ X)
```

```
##      min      Q1  median      Q3     max     mean      sd  n missing
## 5.471029 10.65966 13.04479 21.58549 39.18834 17.02206 9.268847 30      0
```

```
FDh <- 2*(0.215855-10.6597)*(30^(-1/3)); FDh
```

```
## [1] 7.032496
```

```
Scotth <- 3.5*9.26885*(30^(-1/3));Scotth
```

```
## [1] 10.44047
```

```
Sturgesm <- log2(30) + 1; Sturgesm
```

```
## [1] 5.906891
```

```
range(X)
```

```
## [1] 5.471029 39.188344
```

```
# estimate m in an ad hoc way based on exact range (usually goes beyond it a bit)
```

```
maxmin <- 39.18834 - 5.47103
```

```
maxmin/FDh
```

```
## [1] 4.794501
```

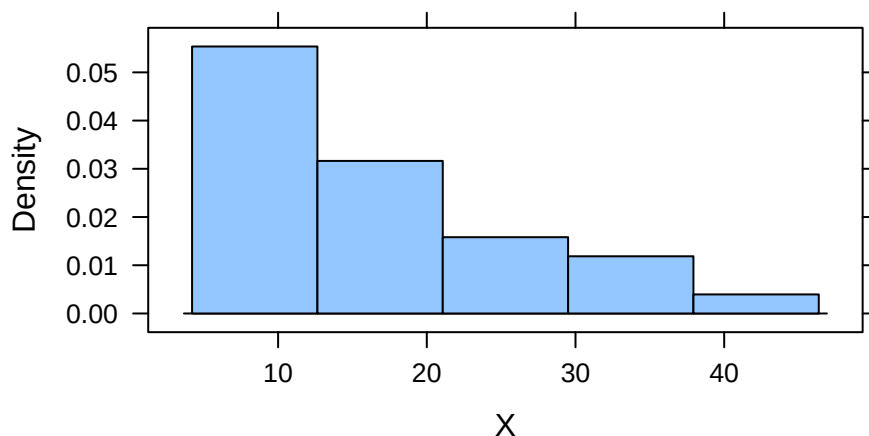
```
maxmin/Scotth
```

```
## [1] 3.229481
```

Sturges suggests 6 bins (though 7 are plotted in R). With the h values for FD and Scott as shown, we get m values of 3.22948 (so four bins) for Scott (plotted), versus 4.7945 (so 5 bins) for FD. FD still plots 7, so some other setting is governing that. This is fine - it means some condition for the endpoints is in force. We shouldn't get fewer bins than these values though. If we want to see what the five bin solution would look like, we can try to get it with nint.

```
histogram(~ X, nint = 5,  
  main = "Ad hoc FD m suggests roughly 5 bins", type = "density")
```

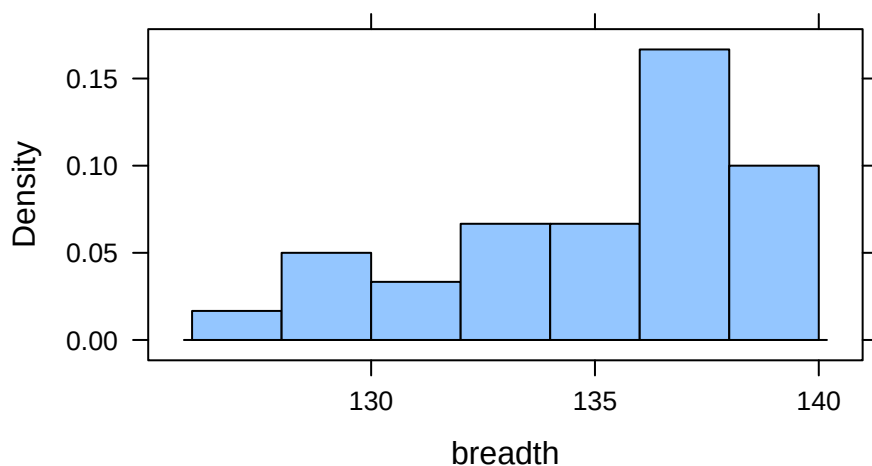
Ad hoc FD m suggests roughly 5 bins



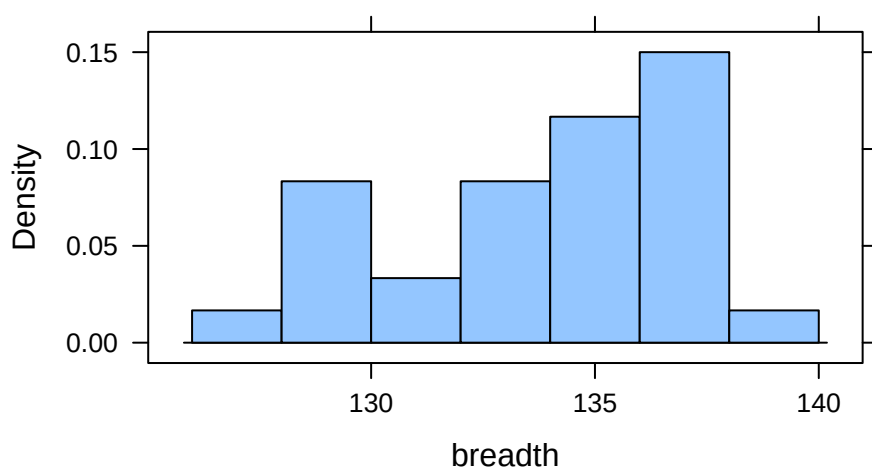
When working on learning the differences here, you will want to solve for the h values and suggested m values, but the pictures don't always do what is suggested. We suggest trying to modify the histograms with nint to get a better sense of the differences. In particular, Sturges and FD when used as options seem to give similar plots a lot, even though their h and m values do not match. This suggests other options in R help govern the visual, but you can still explore with nint.

Other options

```
# change interval to closed, open instead of open, closed
histogram(~ breadth, data = p3,
          breaks = "FD", right = FALSE, type = "density")
```



```
histogram(~ breadth, data = p3,
          breaks = "FD", right = TRUE, type = "density") # default
```



ASH

In order to create univariate average shifted histograms, we need two functions from the **ash** package - **ash1** and **bin1**. There are bivariate versions if you are interested. The **bin** function is called first, and is an input to the **ash** function.

```
favstats(~ breadth, data = p3)
```

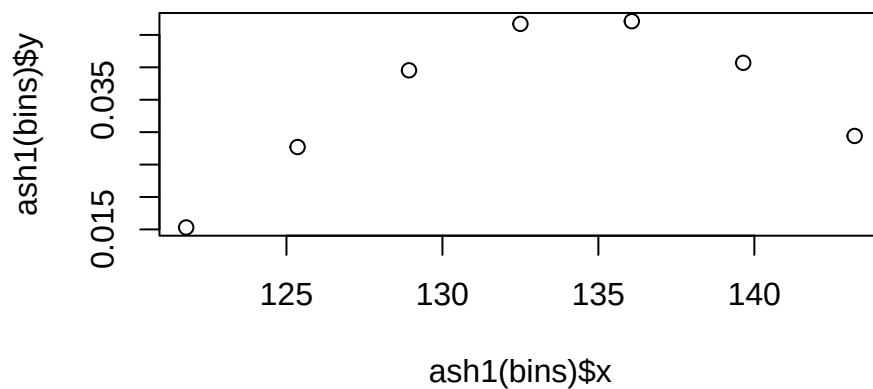
```
##   min      Q1 median  Q3 max    mean      sd  n missing
```

```
## 126 132.25 136 137 140 134.4667 3.481313 30 0
# input data, range of bins/data values, and number of bins
bins <- bin1(p3$breadth, c(120, 145), 7)
# output from the function, really want to plot
ash1(bins)

## [1] "ash estimate nonzero outside interval ab"

## $x
## [1] 121.7857 125.3571 128.9286 132.5000 136.0714 139.6429 143.2143
##
## $y
## [1] 0.01530073 0.02770597 0.03953995 0.04670307 0.04711191 0.04069927 0.02941414
##
## $m
## [1] 5
##
## $ab
## [1] 120 145
##
## $kopt
## [1] 2 2
##
## $ier
## [1] 1
# dot plot of the estimates for each of the 7 bins
plot(ash1(bins))

## [1] "ash estimate nonzero outside interval ab"
```



The `ash` function has more functionality, but it requires understanding kernels, which is in a later section (below).

Understanding Kernels and Density plots

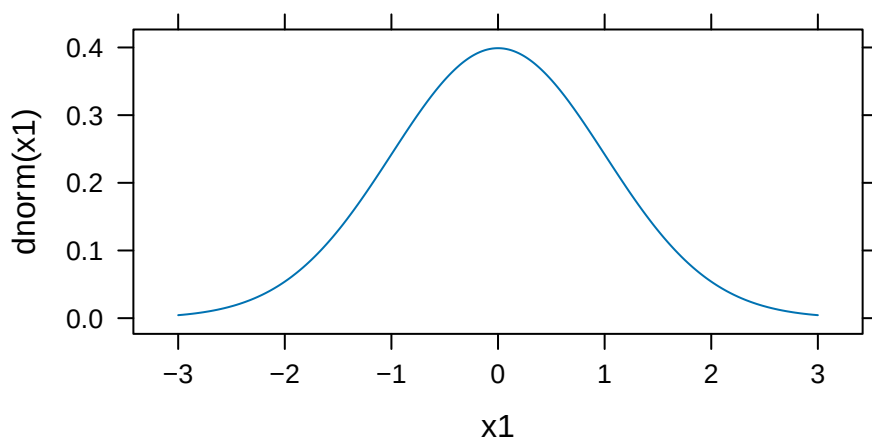
Kernel Plots

These are the plots of the kernels in the book, plus some others (taken from a list of commonly used kernels online.)

```
x1 <- seq(-3, 3, by = 0.01)
x <- seq(-1, 1, by = 0.01)

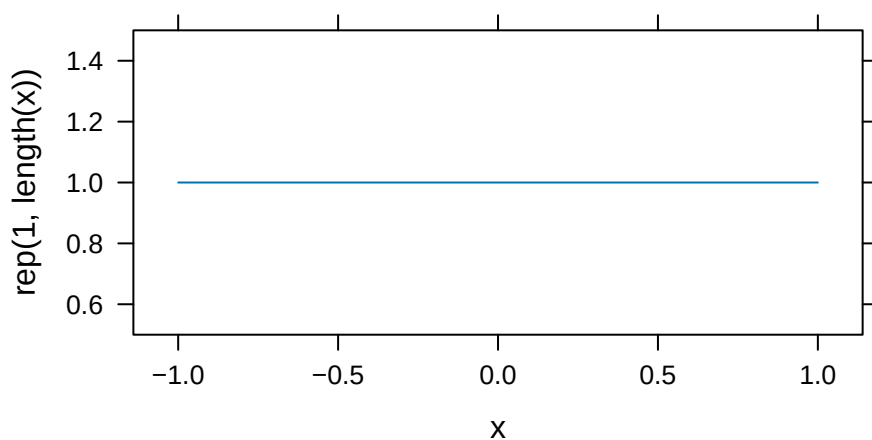
# plotted using lattice graphics for quick visual (gg_formula likes data sets)
xyplot(dnorm(x1) ~ x1, main = "Standard Normal Kernel", type = "l")
```

Standard Normal Kernel



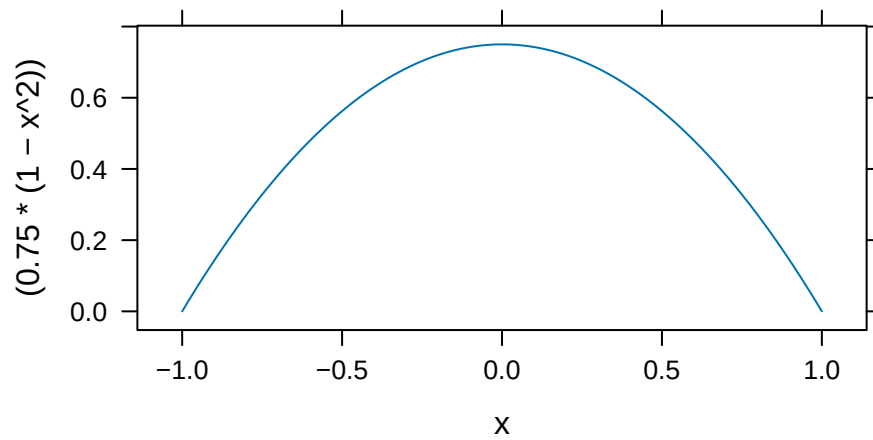
```
xyplot(rep(1, length(x)) ~ x, main = "Box Kernel", type = "l")
```

Box Kernel



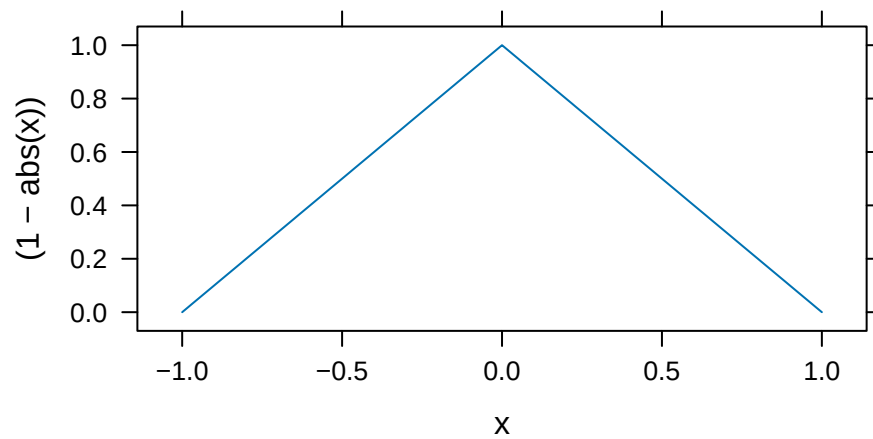
```
xyplot((0.75*(1-x^2)) ~ x, main = "Epanechnikov Kernel", type = "l")
```

Epanechnikov Kernel



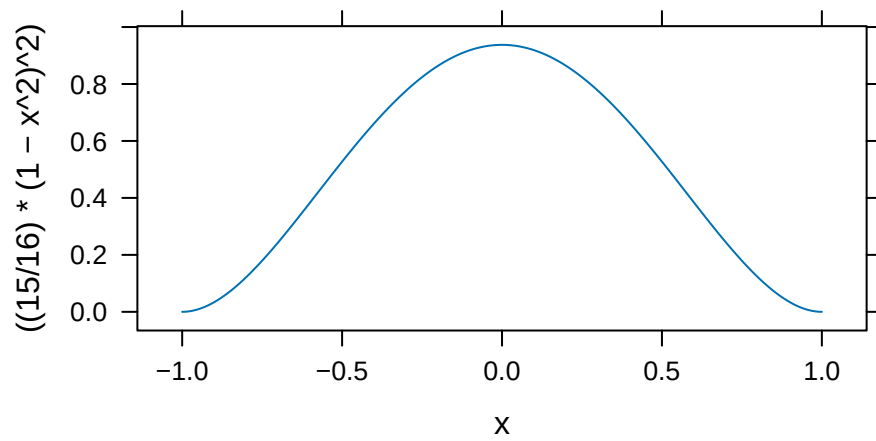
```
xyplot((1-abs(x)) ~ x, main = "Triangle Kernel", type = "l")
```

Triangle Kernel



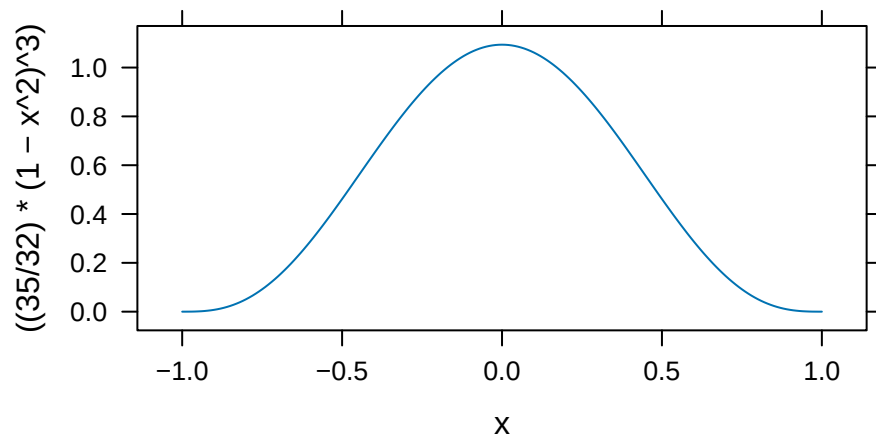
```
xyplot(((15/16)*(1-x^2)^2) ~ x, main = "Quartic Kernel", type = "l")
```

Quartic Kernel



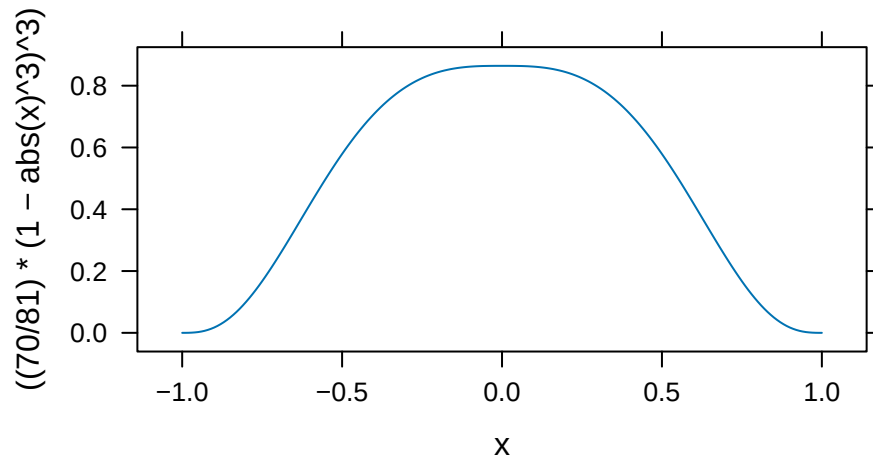
```
xyplot(((35/32)*(1-x^2)^3) ~ x, main = "Triweight Kernel", type = "l")
```

Triweight Kernel



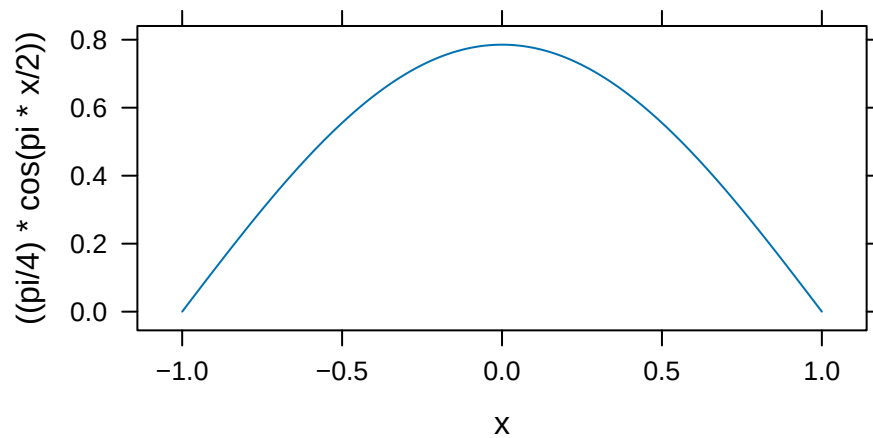
```
xyplot(((70/81)*(1-abs(x)^3)^3) ~ x, main = "Tricube Kernel", type = "l")
```


Tricube Kernel



```
xyplot(((pi/4)*cos(pi*x/2)) ~ x, main = "Cosine Kernel", type = "l")
```

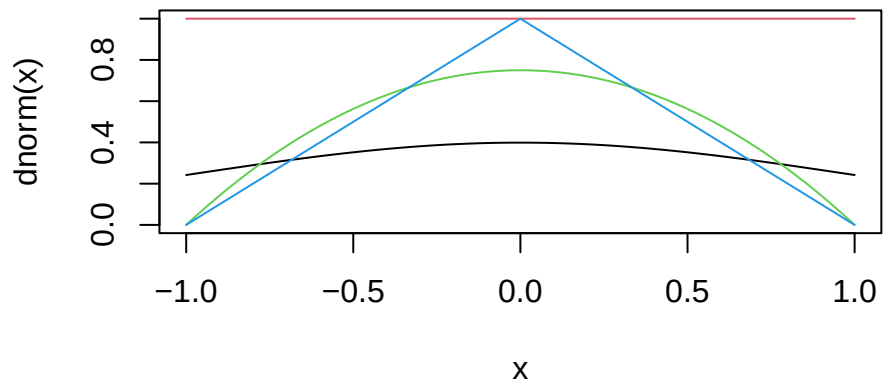
Cosine Kernel



If you want to compare the kernel functions, we can look at a plot with overlaid values. Here are the four kernels from the book overlaid:

```
# standard normal, in black
plot(x, dnorm(x), ylim = c(0,1), main = "Kernel Functions on -1 to 1", type = "l")
lines(x, rep(1, length(x)), col = 2) # box, in red
lines(x, 0.75*(1-x^2), col = 3) # epanechnikov, in green
lines(x, 1-abs(x), col = 4) # triangle, in blue
```

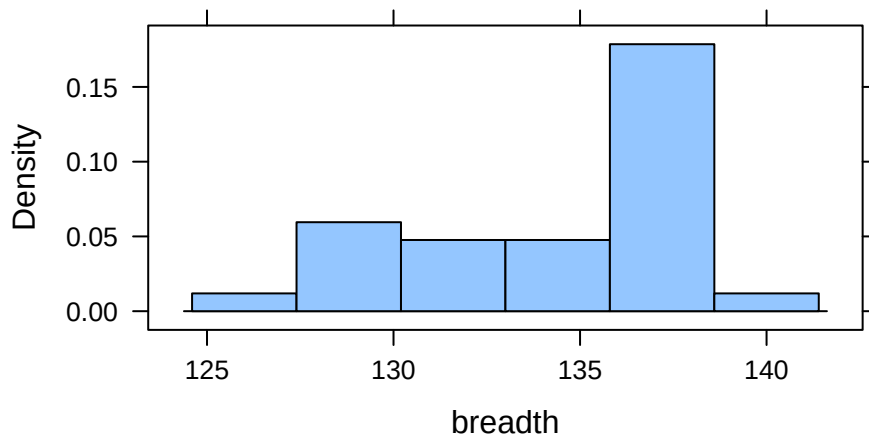
Kernel Functions on -1 to 1



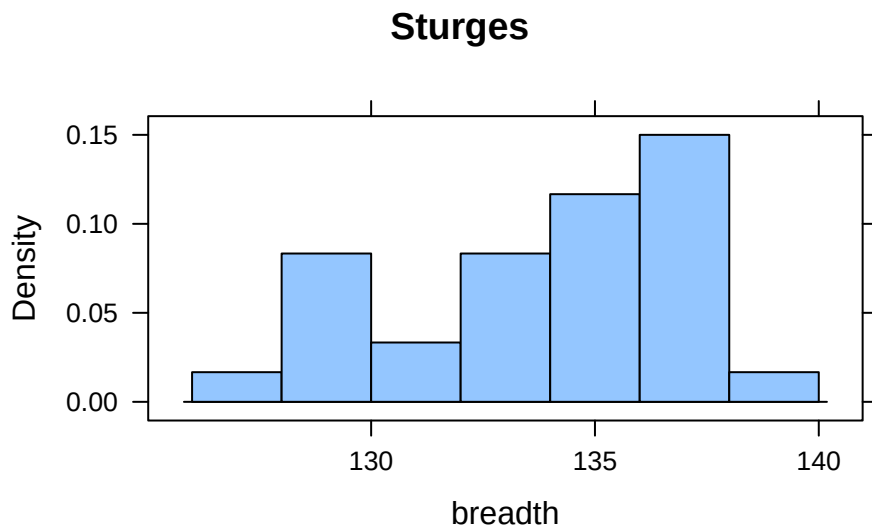
Histogram to Compare two

```
histogram(~ breadth, data = p3, main = "Default") # break default
```

Default



```
histogram(~ breadth, data = p3,  
          breaks = "Sturges", type = "density", main = "Sturges")
```



Density Plots

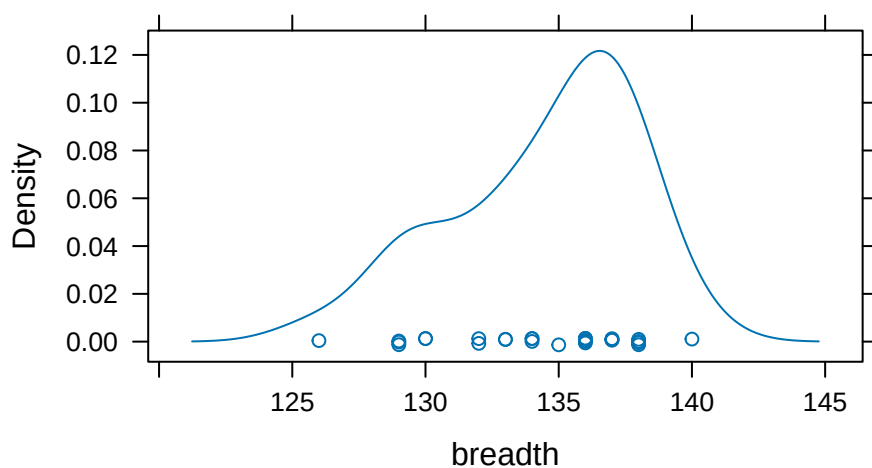
Here are the kernel options in R for the `density` function. This function is called by `densityplot` in order to provide a picture of the estimated density.

kernel options = c("gaussian", "epanechnikov", "rectangular", "triangular", "biweight", "cosine", "optcosine")

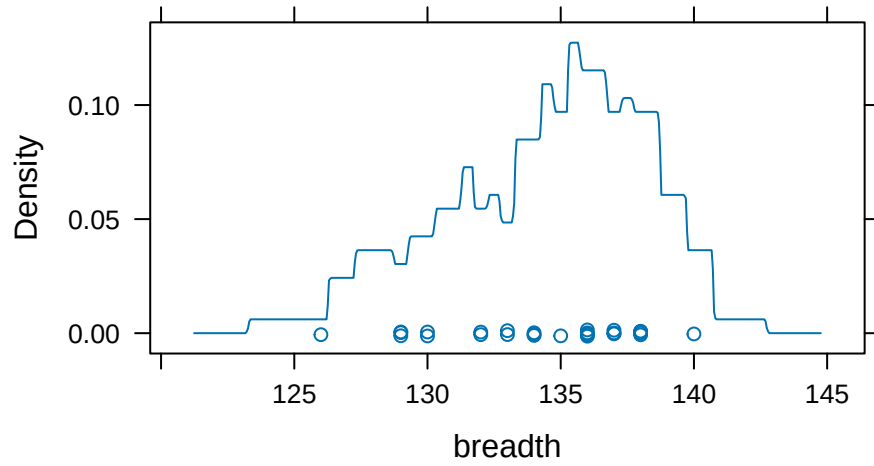
It uses case matching so you only need the first letter of each of these, but you could write out more if you wanted. Gaussian is referring to the normal kernel.

Let's look at some plots.

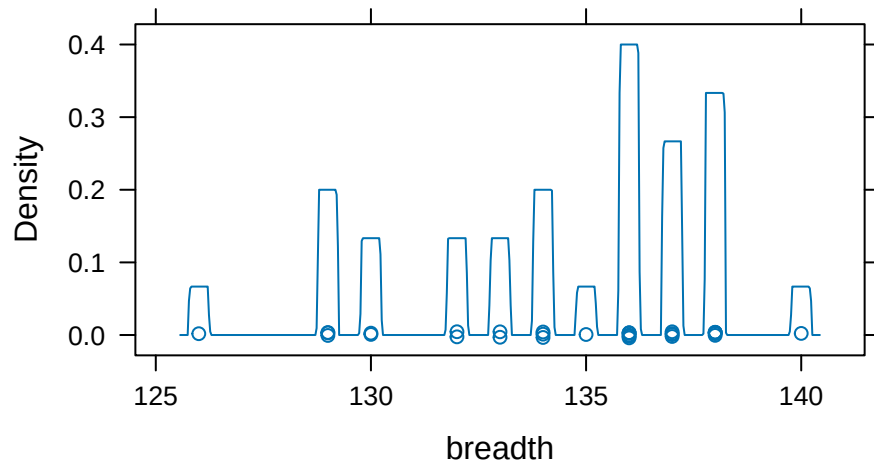
```
# default (uses gaussian kernel, a default bandwidth, and
# evaluates at n=50 points over the range of data values)
densityplot(~ breadth, data = p3)
```



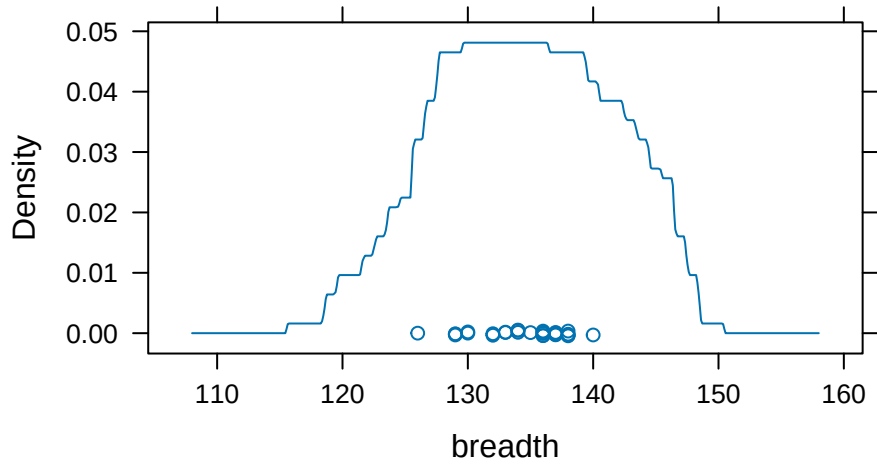
```
densityplot(~ breadth, data = p3, kernel = "r") # box kernel is rectangular
```



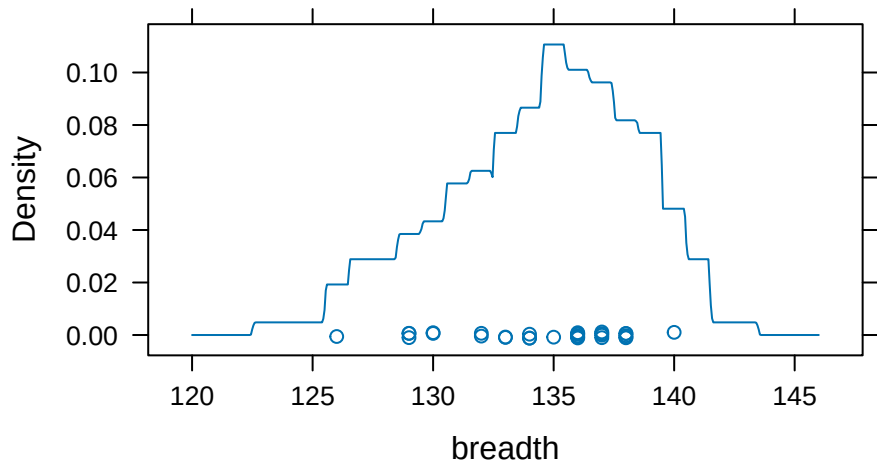
```
densityplot(~ breadth, data = p3, kernel = "r", bw = 1/(4*sqrt(3))) # h=0.5, poor here
```



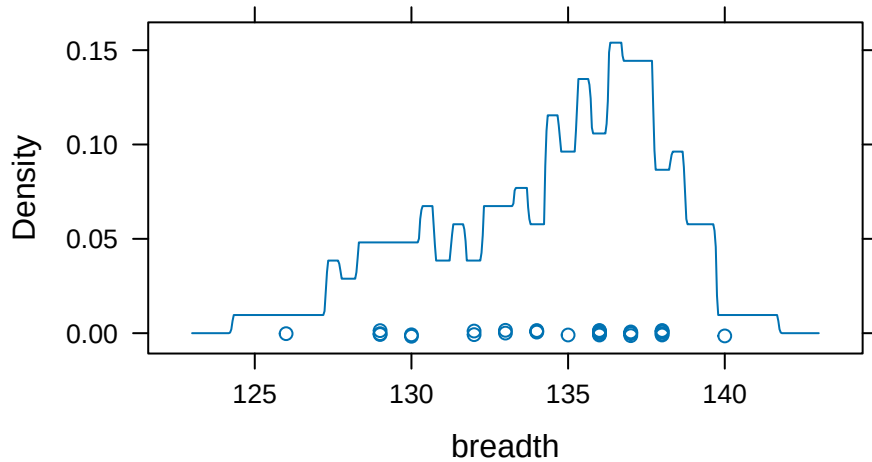
```
densityplot(~ breadth, data = p3, kernel = "r", bw = 6) # oversmoothed
```



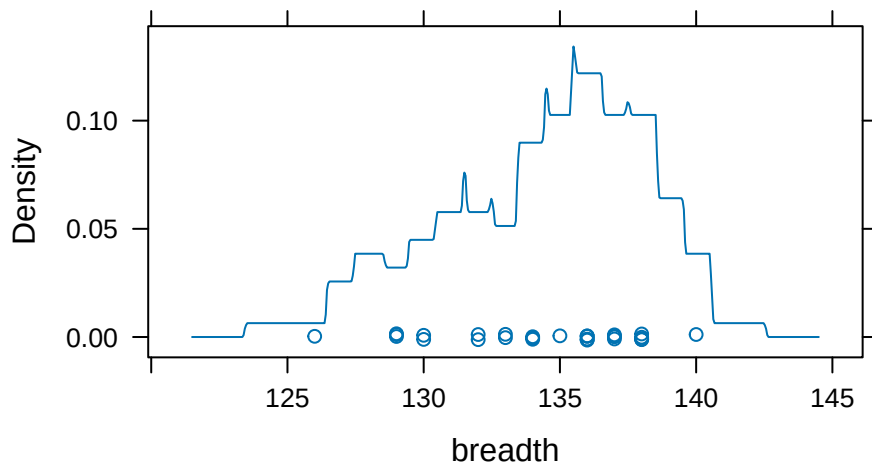
```
densityplot(~ breadth, data = p3, kernel = "r", bw = 2)
```



```
densityplot(~ breadth, data = p3, kernel = "r", bw = 1)
```



```
densityplot(~ breadth, data = p3, kernel = "r", bw = 1.5)
```

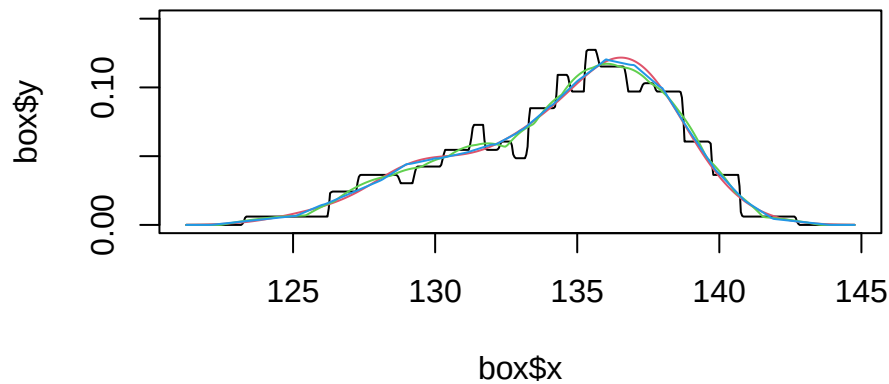


What about viewing this with some other kernels? We will also show you how to save density estimates so you can look at them, or plot them yourself (including in overlays).

```
box <- density(p3$breadth, kernel = "r") # saved box kernel estimate with default h
norm <- density(p3$breadth, kernel = "g") # saved normal kernel "
epan <- density(p3$breadth, kernel = "e") # etc.
tria <- density(p3$breadth, kernel = "t")
```

Now we can do the overlay using the saved values.

```
plot(box$x, box$y, ylim = c(0,0.15), type = "l") # adjust y for scaling
lines(norm$x, norm$y, col = 2)
lines(epan$x, epan$y, col = 3)
lines(tria$x, tria$y, col = 4)
```



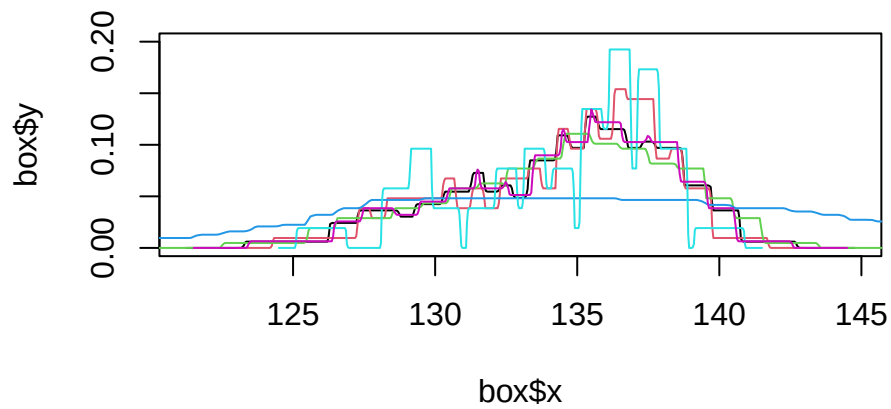
The Effect of Bandwidth

We can investigate the effect of bandwidth through viewing plots.

```
box1 <- density(p3$breadth, kernel = "r", bw = 1)
box2 <- density(p3$breadth, kernel = "r", bw = 2)
box6 <- density(p3$breadth, kernel = "r", bw = 6)
boxa <- density(p3$breadth, kernel = "r", bw = 0.5)
boxb <- density(p3$breadth, kernel = "r", bw = 1.5)
```

Here, we overlay the breadth density estimate using the box kernel with 6 different bandwidths (the default and the 5 we just saved).

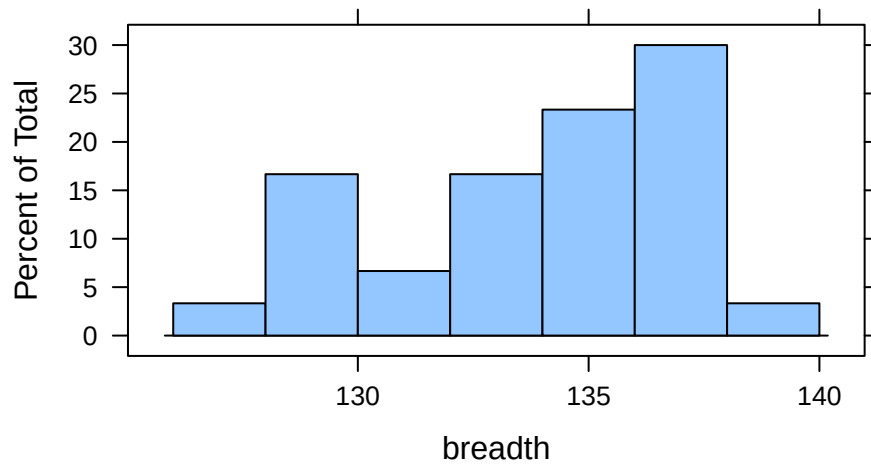
```
plot(box$x, box$y, ylim = c(0,0.2), type = "l") # adjust y for scaling; default - black
lines(box1$x, box1$y, col = 2) # red
lines(box2$x, box2$y, col = 3) # green
lines(box6$x, box6$y, col = 4) # blue (largely flat in middle range)
lines(boxa$x, boxa$y, col = 5) # light blue (highest peaks)
lines(boxb$x, boxb$y, col = 6) # magenta/purple, near black/red (which should make sense)
```



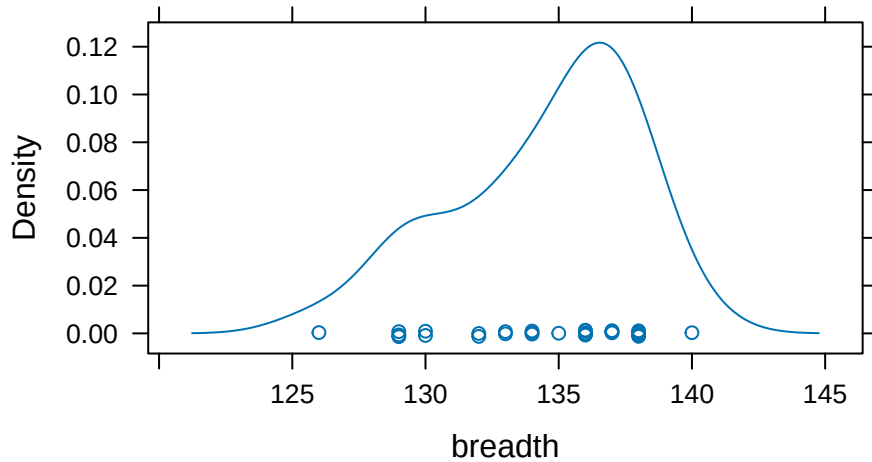
Bandwidth Selection Methods

Reference Pictures and Details

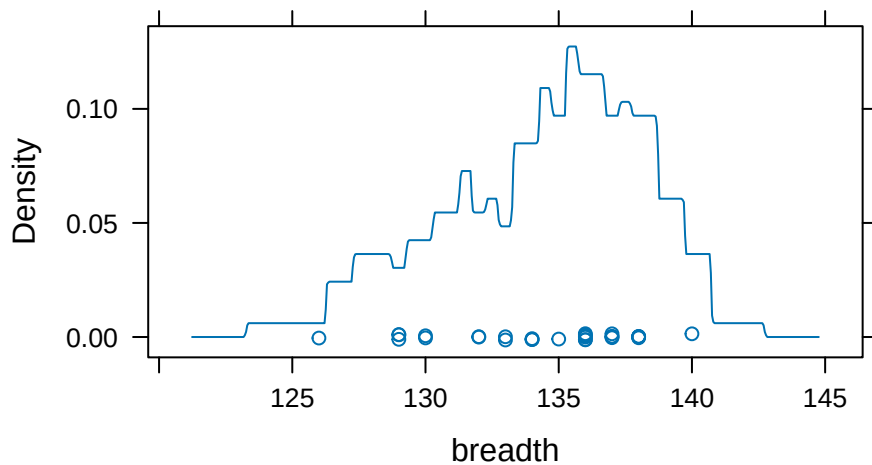
```
histogram(~ breadth, data = p3, breaks = "Sturges")
```



```
# default (uses gaussian kernel, a default bandwidth and evaluates at n=50 points
# over the range of data values) and saves plot
myplot <- densityplot(~ breadth, data = p3)
myplot # prints plot
```

```
densityplot(~ breadth, data = p3, kernel = "r") # box kernel is rectangular
```



```
box <- density(p3$breadth, kernel = "r") # saved box kernel estimate with default h
norm <- density(p3$breadth, kernel = "g") # saved normal kernel "
epan <- density(p3$breadth, kernel = "e")
tria <- density(p3$breadth, kernel = "t")
```

Why would we want to save a plot? In order to see the saved options. Here, unfortunately, it doesn't appear to let us see the chosen bandwidth (bw) value. So, how can we get the bw values actually used? We can look at the `density` function output that was saved. For example:

```
names(box)
```

```
## [1] "x"      "y"      "bw"     "n"      "old.coords"
## [6] "call"   "data.name" "has.na"
```

```
list(box)
```

```
## [[1]]
```

```
##
## Call:
## density.default(x = p3$breadth, kernel = "r")
##
## Data: p3$breadth (30 obs.); Bandwidth 'bw' = 1.587
##
##      x      y
## Min.  :121.2  Min.  :0.000000
## 1st Qu.:127.1  1st Qu.:0.006064
## Median :133.0  Median :0.036381
## Mean   :133.0  Mean   :0.042117
## 3rd Qu.:138.9  3rd Qu.:0.065088
## Max.   :144.8  Max.   :0.127334
box$bw
## [1] 1.586943
```

We can see the bandwidth chosen for the box kernel here was 1.587. We will use this to compare the bandwidths generated according to the different methods (described in Section 12.3 of HWC).

Fixed Bandwidth Comparisons

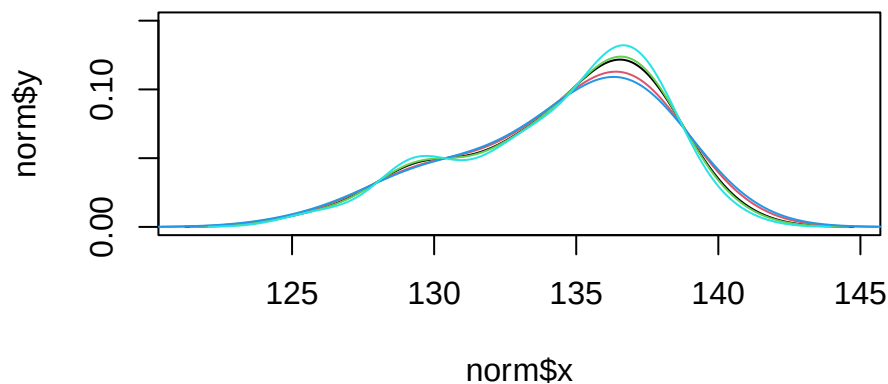
The options for bandwidth (bw) include: “nrd0”, “nrd”, “ucv”, “bcv”, and “SJ”.

```
norm <- density(p3$breadth, kernel = "g", bw = "nrd0") # saved normal kernel, default h
norm2 <- density(p3$breadth, kernel = "g", bw = "nrd")
norm3 <- density(p3$breadth, kernel = "g", bw = "ucv")
norm4 <- density(p3$breadth, kernel = "g", bw = "bcv") # warning obtained

## Warning in bw.bcv(x): minimum occurred at one end of the range
norm5 <- density(p3$breadth, kernel = "g", bw = "SJ")
```

This fits all the density estimates. We can plot them to compare, and also pull out the bandwidths to compare. If you don’t want to plot the curves in color, you can adjust other aspects - make them dashed, etc. Ask for assistance if you want to do that and aren’t sure how to tackle it.

```
plot(norm$x, norm$y, ylim = c(0,0.15), type = "l") # adjust y for scaling
lines(norm2$x, norm2$y, col = 2) # red
lines(norm3$x, norm3$y, col = 3) # green
lines(norm4$x, norm4$y, col = 4) # blue
lines(norm5$x, norm5$y, col = 5) # light blue
```



So, here, we see that all the estimates capture the main peak, but the lesser peak is obscured in some estimates.

```
norm$bw # 1.587
```

```
## [1] 1.586943
```

```
norm2$bw # 1.869
```

```
## [1] 1.869066
```

```
norm3$bw # 1.525
```

```
## [1] 1.525045
```

```
norm4$bw # 2.01
```

```
## [1] 2.01002
```

```
norm5$bw # 1.313
```

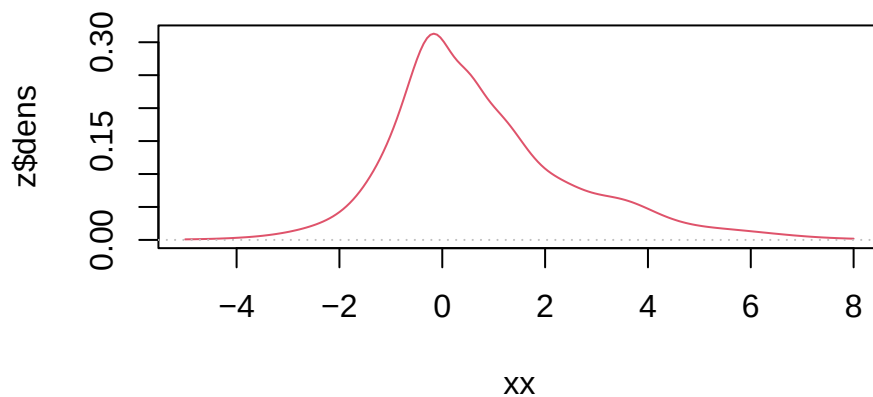
```
## [1] 1.31313
```

Again, for the normal kernel, the bw value *is* the h value. For other kernels, there is a relationship, but it was only specified for the box/rectangular kernel in your text. If you are asked to compare bandwidths, just compare the bw values, knowing that R relates computations of h to bw selections.

Variable Bandwidth

This is the example provided for the function `akj` on adaptive kernel estimation. The underlying data are a mixture of two standard normal distributions. You have to create a vector where you want the adaptive h's to be constructed, and both the data and that vector must be input to the `akj` function.

```
set.seed(1)
x <- c(rnorm(600), 2 + 2*rnorm(400)) # data
xx <- seq(-5, 8, length = 200) # where to compute the adaptive h values
z <- akj(x, xx) # get the density estimate
plot(xx, z$dens, ylim = range(0, z$dens), type = "l", col = 2)
abline(h = 0, col = "gray", lty = 3)
```



Here is an example using our data:

```
favstats(~ breadth, data = p3)
```

```
## min      Q1 median  Q3 max      mean      sd  n missing
## 126 132.25   136 137 140 134.4667 3.481313 30      0
```

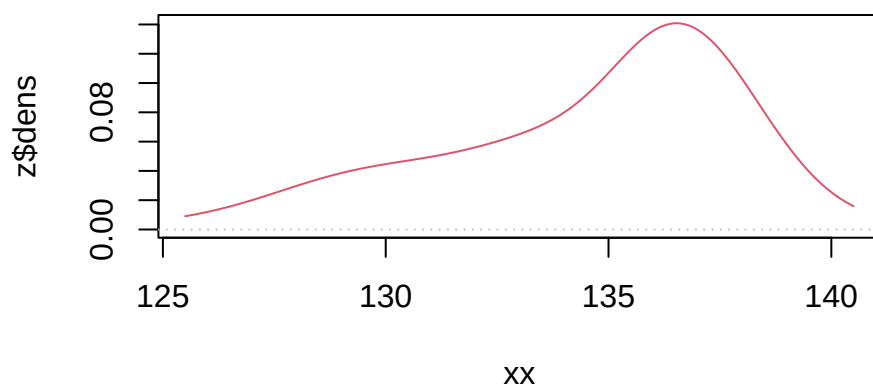
```
# 200 equally spaced points between 125.5 and 140.5
```

```
xx <- seq(125.5, 140.5, length = 200)
```

```
z <- akj(p3$breadth, xx)
```

```
plot(xx, z$dens, ylim = range(0, z$dens), type = "l", col = 2)
```

```
abline(h = 0, col = "gray", lty = 3)
```



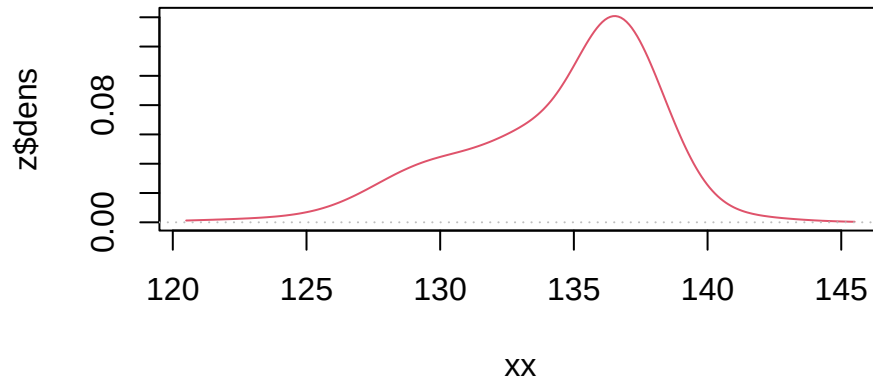
This would look more clean (look at the edges, they haven't dropped down near 0 yet) if the xx range was extended a bit more (beyond the data range, so the edges drop).

```
# as you increase the length, computing time will increase
```

```
xx <- seq(120.5, 145.5, length = 250)
```

```
z <- akj(p3$breadth, xx)
```

```
plot(xx, z$dens, ylim = range(0, z$dens), type = "l", col = 2)
abline(h = 0, col = "gray", lty = 3)
```



Data Sources

The Egyptian skull dataset is a subset of a dataset from Thomson, A. and Randall-Maciver, R. (1905) *Ancient Races of the Thebaid*, Oxford: Oxford University Press. Also found in: Hand, D.J., et al. (1994) *A Handbook of Small Data Sets*, New York: Chapman & Hall, pp. 299-301. Manly, B.F.J. (1986) *Multivariate Statistical Methods*, New York: Chapman & Hall.

Standard Reference

These materials were created for use in an undergraduate nonparametrics course. Where provided, textbook references refer to *Nonparametric Statistical Methods* (3rd Edition) by Hollander, Wolfe, and Chicken. Notation is chosen to be consistent with that text. However, materials may be used independently of the textbook.

License

This work is protected by creative commons license CC BY-NC-SA.